

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное образовательное учреждение высшего образования
САНКТ – ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

старший преподаватель

должность, уч. степень, звание

подпись, дата

Д.О. Шевяков

инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №12

Отношения между объектами

по курсу: Основы программирования

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № 4321

подпись, дата

Г.В.Буренков

инициалы, фамилия

Санкт-Петербург 2024

СОДЕРЖАНИЕ

1 Цель работы.....	3
2 Задание.....	3
3 Описание работы программы	4
4 Результаты работы программы.....	10
5 Вывод	11
Приложение А. Листинг Программы.....	12

1 Цель работы

Проанализируйте отношения между классами в приложении для управления автопарком и изобразите их на диаграмме классов. Используйте следующие типы отношений: наследование, ассоциация, агрегация, композиция, зависимость, реализация.

2 Задание

Опишите каждый тип отношения между классами, приведя примеры из вашего приложения. Укажите семантику каждого отношения и графическое представление на диаграмме классов.

3 Описание работы программы

Композиция и агрегация являются важными концепциями в объектно-ориентированном программировании, которые помогают описать отношения между классами. В случае с классом Car, он содержит компоненты, такие как Engine и Wheel, что иллюстрирует композицию "часть-целое". Двигатель и колеса не могут существовать независимо от автомобиля, что подчеркивает их неразрывную связь.. Также стоит отметить, что Car может содержать FuelTank, который, в свою очередь, не может быть использован в другом автомобиле без специальной операции.

Ассоциация между классами также играет важную роль в моделировании отношений. Например, CarPark имеет отношение "многие ко многим" с Car, что означает, что один автопарк может содержать несколько автомобилей, а один автомобиль может принадлежать только одному автопарку. В то же время, Car может иметь отношение "один ко многим". На рисунке 1 изображена композиция и агрегация классов.

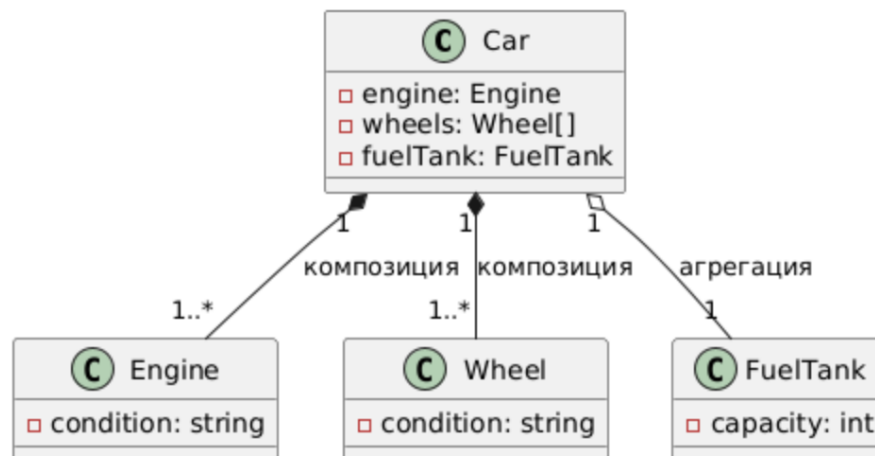


Рисунок 1 – Композиция и агрегация

Зависимости между классами также важны для понимания их взаимодействия. Класс Car зависит от Info, так как он использует методы этого класса для получения информации об автомобиле. Также Car может зависеть от Repair, если такой класс существует, поскольку ремонтные

работы могут выполняться над автомобилем. Важно отметить, что Engine зависит от Fuel, так как двигатель использует топливо для своей работы, а Car также зависит от FuelTank, что подчеркивает важность этих компонентов в функционировании автомобиля. На рисунке 2 изображена ассоциация.

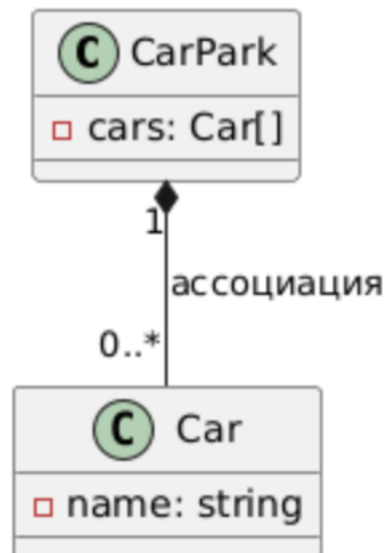


Рисунок 2 – Ассоциация классов

Агрегация, в отличие от композиции, позволяет классам существовать независимо друг от друга. Например, класс Car может агрегировать класс Engine, что означает, что двигатель может быть использован в нескольких автомобилях или храниться отдельно на складе. Это позволяет реализовать возможность замены двигателя в приложении, что добавляет гибкости в управление автомобилем. Таким образом, двигатель, который был снят с одного автомобиля, может быть установлен в другой, что делает систему более универсальной. На рисунке 3 изображена зависимость классов.

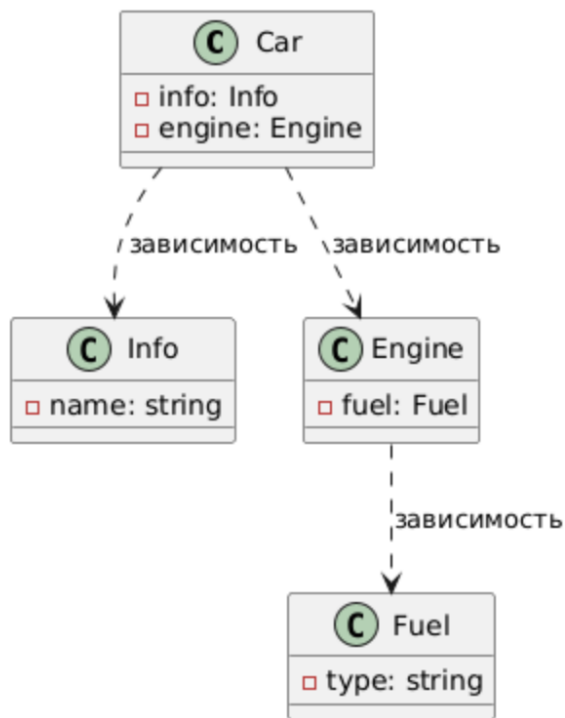


Рисунок 3 – Зависимость классов

Интерфейсы также играют важную роль в определении общих методов для классов. Класс Car может реализовать интерфейс IVehicle, который может включать в себя методы, такие как StartEngine и StopEngine. Это позволяет создать единый стандарт для всех транспортных средств, что упрощает взаимодействие между различными классами и их реализациями. Интерфейсы обеспечивают гибкость и возможность расширения функциональности без изменения существующего кода. На рисунке 4 изображены интерфейсы классов.

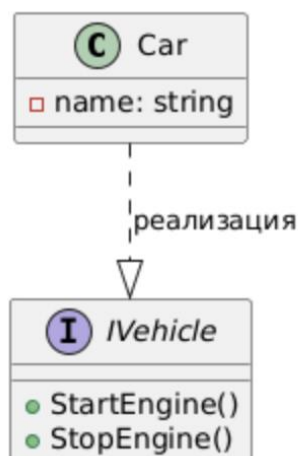


Рисунок 4 – Интерфейсы классов

Таким образом, отношения между классами Car, Engine, Wheel, FuelTank и другими компонентами создают сложную, но логичную структуру, которая отражает реальную жизнь. Композиция, агрегация, ассоциация и зависимости помогают разработчикам лучше понять, как различные элементы взаимодействуют друг с другом. Это знание позволяет создавать более эффективные и устойчивые системы, которые могут адаптироваться к изменениям и требованиям пользователей.

На рисунке 5 изображена диаграмма классов программы.

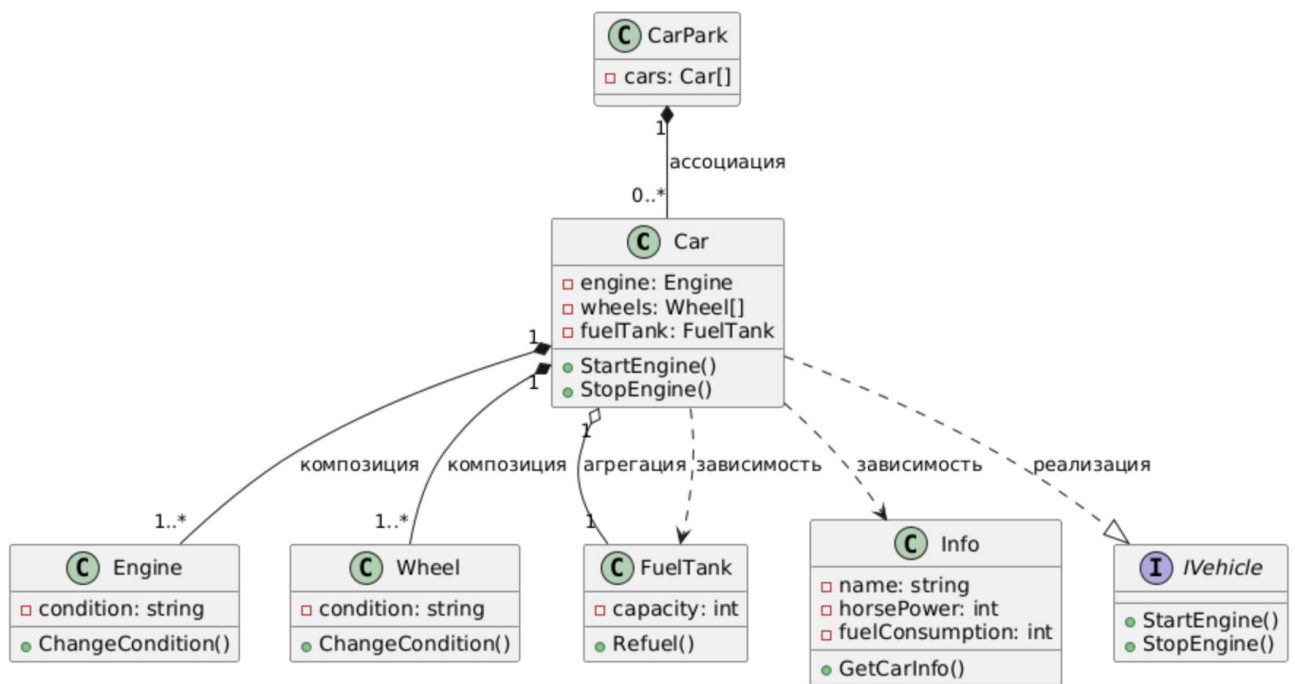


Рисунок 5 – Диаграмма классов

На рисунке 6 представлена структура оформления базового окна Windows Forms.

The screenshot shows a Windows Forms application window titled "Car Management". The window has a standard Windows title bar with minimize, maximize, and close buttons. The main area contains several input fields and buttons. On the left, there are four labels with corresponding text boxes: "Введите имя авто:", "Введите мощность авто:", "Введите расход авто:", and "Введите бак авто:". Below these are two buttons: "Создать авто" and "Удалить авто". At the bottom left, there is a checkbox labeled "Подробно" and a button labeled "Показать". In the center, there is a large empty rectangular box. On the right side, there are four buttons stacked vertically: "Заправка", "Передвижение", "Запустить", and "Выключить". Below these is a button labeled "Замениь". At the top right, there is a checkbox labeled "Высокая скорость".

Рисунок 6 – Оформление окна Windows Forms

4 Результаты работы программы с примерами

На рисунке 7 представлен результат работы программы показа работы интерфейсов компонента.

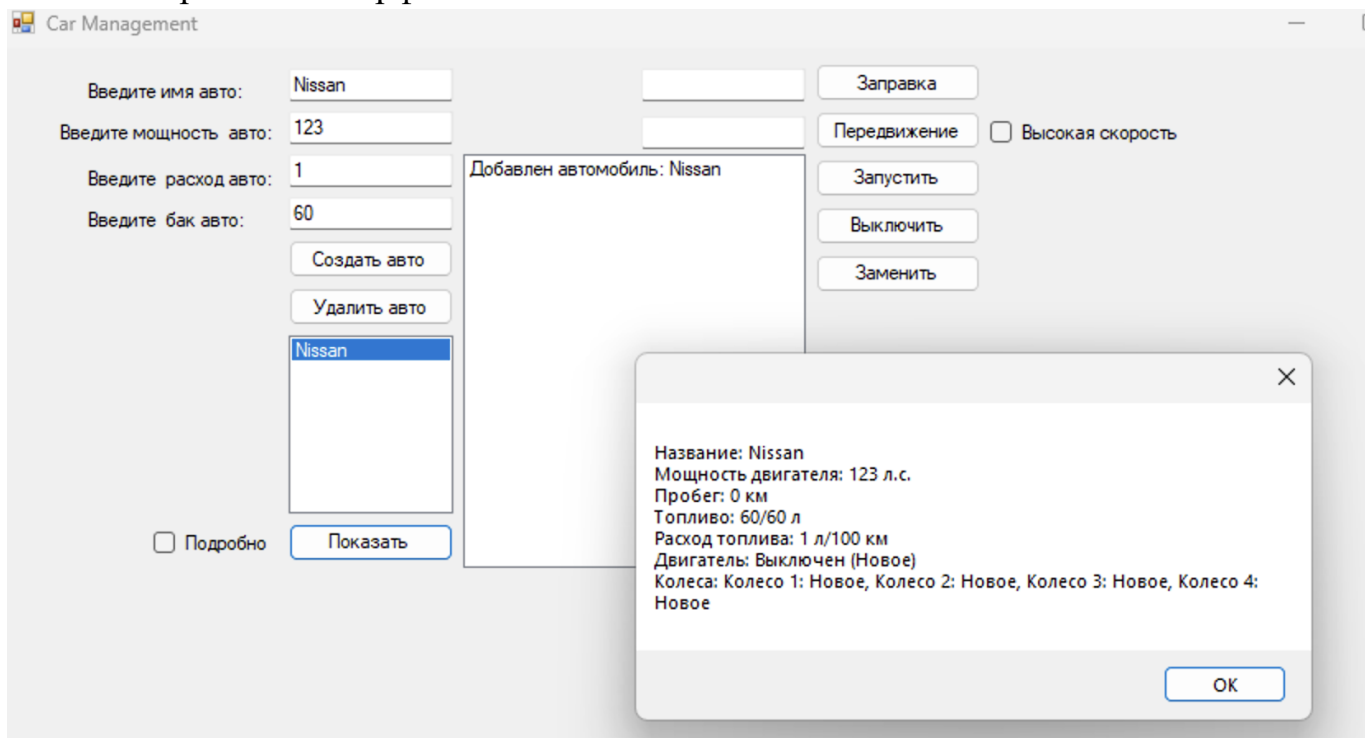


Рисунок 7 – Результат работы программы

5 Вывод

В ходе выполнения лабораторной работы была изучена концепция отношений между классами в объектно-ориентированном программировании на языке C#. Были рассмотрены основные типы отношений: наследование, ассоциация, агрегация, композиция, зависимость и реализация.

Анализ приложения для создания и управления автопарком позволил увидеть реализацию этих отношений на практике. В частности, было выявлено, что приложение использует композицию для связи автомобиля с двигателем и колесами, агрегацию для связи автомобиля с топливным баком, ассоциацию для связи автопарка с автомобилями и зависимости между классами для реализации функциональности приложения.

Изучение различных типов отношений между классами является важным этапом в освоении объектно-ориентированного программирования. Это позволяет создавать более структурированный и гибкий код, улучшать повторное использование кода и упрощать поддержание и развитие программного обеспечения.

Приложение А. Листинг Программы

Файл «Component»

```
using System;

public abstract class Component
{
    // Абстрактный класс Component, который реализует общие методы и события для всех компонентов автомобиля
    // Свойство Condition хранит текущее состояние компонента, по умолчанию "Новое"
    public string Condition { get; protected set; } = "Новое";

    // Метод ChangeCondition для изменения состояния компонента
    // Проверяет, что состояние может быть только "Новое" или "Сломано"
    // При изменении состояния на "Сломано" вызывает событие поломки компонента
    public void ChangeCondition(string condition)
    {
        if (condition != "Новое" && condition != "Сломано")
            throw new ArgumentException("Неверное состояние компонента");

        Condition = condition;

        // Если состояние компонента "Сломано", генерируем событие поломки
        if (Condition == "Сломано")
        {
            OnComponentBroken();
        }
    }

    // Защитный метод для вызова события поломки компонента
    // Срабатывает, если компонент сломался
    protected virtual void OnComponentBroken()
    {
        ComponentBroken?.Invoke(this, EventArgs.Empty); // Генерация события ComponentBroken
    }

    // Событие поломки компонента, которое оповещает об изменении состояния на "Сломано"
```

Файл «IComponent»

```
using System;

public interface IComponent
{
    // Свойство Condition представляет текущее состояние компонента (например, "Новое", "Сломано")
    string Condition { get; set; }

    // Метод для изменения состояния компонента
    // Принимает строку, которая указывает новое состояние компонента (например, "Сломано" или "Новое")
    void ChangeCondition(string condition);
}
```

Файл «Info»

```
using System.Linq;

namespace CarManagementApp
{
    public sealed class Info : Car
    {

```

```

// Конструктор для класса Info, который вызывает конструктор базового класса
public Info(string name, int horsepower, int fuelConsumption, int fuelTankCapacity)
    : base(name, horsepower, fuelConsumption, fuelTankCapacity)
{
}

// Переопределяем метод получения информации о машине
public override string GetCarInfo(bool showDetailed = false)
{
    string info = $"Название: {Name}\n" +
        $"Мощность двигателя: {Engine.HorsePower} л.с.\n" +
        $"Пробег: {Mileage} км\n" +
        $"Топливо: {Fuel}/{FuelTankCapacity} л\n" +
        $"Расход топлива: {FuelConsumption} л/100 км\n" +
        $"Двигатель: {(IsEngineOn ? "Включен" : "Выключен")} ({Engine.Condition})\n" +
        $"Колеса: {string.Join(", ", Wheels.Select((w, i) => $"Колесо {i + 1}: {w.Condition}"))}";

    if (showDetailed)
    {
        info += $"Оставшийся пробег до поломки двигателя: {1000 - Mileage % 1000} км";
        for (int i = 0; i < Wheels.Length; i++)
        {
            info += $"Колесо {i + 1}: осталось до поломки {200 - Mileage % 200} км";
        }
    }

    return info;
}
}
}

```